

Python → Java Cheat Sheet

Core data structures for LeetCode & interviews

1 Dynamic Array

PYTHON

```
nums = [1, 2, 3]
nums.append(4)
```

JAVA

```
List<Integer> nums = new ArrayList<>();
nums.add(1);
nums.add(2);
```

- ArrayList is the closest equivalent to Python's list.
- Supports dynamic resizing.

2 Stack

PYTHON

```
stack = []
stack.append(1)
top = stack.pop()
```

JAVA (RECOMMENDED)

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
int top = stack.pop();
```

ALTERNATIVE (NOT RECOMMENDED)

```
Stack<Integer> stack = new Stack<>();
```

- Most devs prefer Deque/ArrayDeque over Stack.
- push() adds to top; pop() removes & returns top.

3 Queue

PYTHON

```
from collections import deque
q = deque()
q.append(1)
x = q.popleft()
```

JAVA

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(1);
int x = q.poll();
```

- offer() inserts; poll() removes from the front.
- ArrayDeque is generally preferred over LinkedList.

4 Hash Map

PYTHON

```
freq = {}
freq[1] = 5
```

JAVA

```
Map<Integer, Integer> freq = new HashMap<>();
freq.put(1, 5);
```

5 Hash Set

PYTHON

```
seen = set()
seen.add(1)
```

JAVA

```
Set<Integer> seen = new HashSet<>();
seen.add(1);
```

6 Min Heap

PYTHON

```
import heapq
heap = []
heapq.heappush(heap, 5)
smallest = heapq.heappop(heap)
```

JAVA

```
PriorityQueue<Integer> heap =
    new PriorityQueue<>();
heap.offer(5);
int smallest = heap.poll();
```

- Java's PriorityQueue is a min heap by default.

Common LeetCode Starter Template

```
import java.util.*;

class Solution {
    public void example() {
        List<Integer> arr      = new ArrayList<>();
        Deque<Integer> stack   = new ArrayDeque<>();
        Queue<Integer> queue   = new ArrayDeque<>();
        Map<Integer, Integer> map = new HashMap<>();
        Set<Integer> set        = new HashSet<>();
        PriorityQueue<Integer> heap = new PriorityQueue<>();
    }
}
```

Python → Java Mapping

list	→	ArrayList
stack (list)	→	ArrayDeque
deque	→	ArrayDeque
dict	→	HashMap
set	→	HashSet
heapq	→	PriorityQueue

The 90% Toolkit

```
List<Integer> arr = new ArrayList<>();
Deque<Integer> stack = new ArrayDeque<>();
Queue<Integer> queue = new ArrayDeque<>();
Map<Integer, Integer> map = new HashMap<>();
Set<Integer> set = new HashSet<>();
PriorityQueue<Integer> heap =
    new PriorityQueue<>();
```